DSA Streak: Daily LeetCode in Java

2026-11-11 | 5 problems

#	Problem	Difficulty	Topic
1	Combination Sum	Medium	Array, Backtracking
2	Diameter of Binary Tree	Easy	Tree, Depth-First Search
3	Longest Substring Without Repeating Characters	Medium	Hash Table, String
4	Merge Intervals	Medium	Array, Sorting
5	Trapping Rain Water	Hard	Array, Two Pointers

1. Combination Sum

Medium | #39 | Array, Backtracking
https://leetcode.com/problems/combination-sum/

Problem

You get an array of distinct positive integers (candidates) and a target. Return every unique combination of candidates that adds up to target. The same number can be used any number of times. Two combinations are the same if they use the same numbers the same number of times, so [2,2,3] and [3,2,2] count once. Any order is fine.

Examples

Example 1	
Input	`candidates = [2,3,6,7], target = 7`
Output	`[[2,2,3],[7]]`
Why	2 + 2 + 3 = 7 (2 is reused) and 7 = 7. No other combination works.

Example 2	
Input	`candidates = [2,3,5], target = 8`
Output	`[[2,2,2,2],[2,3,3],[3,5]]`
Why	These are the only three multisets that sum to 8.

Example 3	
Input	`candidates = [2], target = 1`
Output	`[]`
Why	2 is already bigger than 1, so nothing can be formed.

Intuition

Build combinations one number at a time and undo the choice after exploring it (backtracking). To avoid duplicates like [2,3] and [3,2], only allow picking numbers at index j or later than the last pick, so every combination is generated in non-decreasing order exactly once. Passing j (not j + 1) into the recursion is what lets a number be reused. Sorting first lets us stop the loop as soon as a number is bigger than what is left.

Approach

1. Sort candidates.
2. Call dfs(a, i = 0, rem = target, cur = empty list).
3. In dfs: if rem == 0, copy cur into res and return.
4. Loop j from i to the end. If a[j] > rem, break (everything after is even bigger).
5. Add a[j] to cur, recurse with dfs(a, j, rem - a[j], cur) so a[j] can be used again, then remove the last element (backtrack).
6. Return res.

Dry run on Example 1

- a = [2,3,6,7], dfs(i=0, rem=7, cur=[])
- j=0: cur=[2], dfs(0, 5) -> j=0: cur=[2,2], dfs(0, 3) -> j=0: cur=[2,2,2], dfs(0, 1): a[0]=2 > 1, break
- back in dfs(0, 3): pop -> [2,2]; j=1: cur=[2,2,3], dfs(1, 0): rem == 0, add [2,2,3] to res; pop; j=2: 6 > 3, break
- back in dfs(0, 5): pop -> [2]; j=1: cur=[2,3], dfs(1, 2): 3 > 2, break; pop; j=2: 6 > 5, break
- back in root: pop -> []; j=1: cur=[3], dfs(1, 4) -> j=1: cur=[3,3], dfs(1, 1): 3 > 1 break; pop; j=2: 6 > 4 break
- root j=2: cur=[6], dfs(2, 1): 6 > 1 break; pop

- root j=3: cur=[7], dfs(3, 0): rem == 0, add [7]; pop
- res = [[2,2,3],[7]]

Java solution

```
import java.util.*;

class Solution {
    List<List<Integer>> res = new ArrayList<>();

    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);
        dfs(candidates, 0, target, new ArrayList<>());
        return res;
    }

    void dfs(int[] a, int i, int rem, List<Integer> cur) {
        if (rem == 0) {
            res.add(new ArrayList<>(cur));
            return;
        }
        for (int j = i; j < a.length; j++) {
            if (a[j] > rem) break;
            cur.add(a[j]);
            dfs(a, j, rem - a[j], cur);
            cur.remove(cur.size() - 1);
        }
    }
}
```

Complexity

Time: O(N^(T/M + 1)) in the worst case, where N = number of candidates, T = target, M = smallest candidate (depth of the tree is at most T/M) Space: O(T/M) for the recursion stack and cur, not counting the output

Edge cases handled

- Smallest candidate already bigger than target: returns an empty list.
- Single candidate that divides target, e.g. [2] with target 40: one combination of twenty 2s (deepest recursion).
- Unsorted input: sorting first keeps the early break correct.
- Remember to add a copy (new ArrayList<>(cur)), not cur itself, or every result ends up empty after backtracking.

Pattern to remember

Combinations with reuse: backtrack with a start index, recurse with the same index j to allow reuse (j + 1 when each item can be used once), sort and break early when the item exceeds the remainder.

2. Diameter of Binary Tree

Easy | #543 | Tree, Depth-First Search, Binary Tree
https://leetcode.com/problems/diameter-of-binary-tree/

Problem

Given the root of a binary tree, return the length (number of edges) of the longest path between any two nodes. The path does not have to pass through the root.

Examples

Example 1	
Input	`root = [1,2,3,4,5]`
Output	`3`
Why	The longest path is 4 -> 2 -> 1 -> 3 (or 5 -> 2 -> 1 -> 3), which has 3 edges.

Example 2	
Input	`root = [1,2]`
Output	`1`
Why	Only one edge, 1 - 2.

Intuition

Any path in a tree has one highest node where it turns. If the path turns at node X, its best length is (depth of X's left subtree) + (depth of X's right subtree). So compute depths bottom up, and at every node try l + r as a candidate answer. One DFS gives both the depth to return upward and the diameter candidate.

Approach

1. Keep a field ans = 0.
2. depth(node): if node is null return 0.
3. l = depth(node.left), r = depth(node.right).
4. ans = max(ans, l + r): the longest path that turns at this node.
5. Return 1 + max(l, r), the height of this subtree counted in nodes.
6. Call depth(root) and return ans.

Dry run on Example 1

- Tree: 1 has children 2 and 3; 2 has children 4 and 5.
- depth(4): l=0, r=0, ans=max(0,0)=0, returns 1
- depth(5): l=0, r=0, ans stays 0, returns 1
- depth(2): l=1, r=1, ans=max(0,2)=2, returns 2
- depth(3): l=0, r=0, ans stays 2, returns 1
- depth(1): l=2, r=1, ans=max(2,3)=3, returns 3
- Return ans = 3

Java solution

```
class Solution {
    int ans = 0;

    public int diameterOfBinaryTree(TreeNode root) {
        depth(root);
        return ans;
    }

    int depth(TreeNode node) {
        if (node == null) return 0;
```

```
        int l = depth(node.left);
        int r = depth(node.right);
        ans = Math.max(ans, l + r);
        return 1 + Math.max(l, r);
    }
}
```

Complexity

Time: O(n), each node visited once Space: O(h) recursion stack, where h is the tree height (O(n) for a skewed tree)

Edge cases handled

- Single node: no edges, answer 0.
- Longest path not through the root, e.g. a deep left subtree with two long branches: the ans update at every node handles it.
- Skewed tree of 10^4 nodes: recursion is 10^4 deep, fine on LeetCode.
- Negative values do not matter, only the shape counts.

Pattern to remember

Tree path problems: DFS that returns one thing upward (height) while updating a global answer with a combination of both children (l + r). Same pattern as Binary Tree Maximum Path Sum.

3. Longest Substring Without Repeating Characters

Medium | #3 | Hash Table, String, Sliding Window
https://leetcode.com/problems/longest-substring-without-repeating-characters/

Problem

Given a string s, find the length of the longest substring (contiguous) that has no repeated characters. s can contain letters, digits, symbols and spaces.

Examples

Example 1	
Input	`s = "abcabcbb"`
Output	3
Why	The answer is "abc", with length 3.

Example 2	
Input	`s = "bbbbb"`
Output	1
Why	The answer is "b", with length 1.

Example 3	
Input	`s = "pwwkew"`
Output	3
Why	The answer is "wke", with length 3. "pwke" is a subsequence, not a substring.

Intuition

Keep a window [l, r] with all distinct characters and grow r one step at a time. When s[r] was already seen inside the window, the window must start right after that earlier position. Storing last[c] = (last index of c) + 1 lets us jump l there directly instead of shrinking one by one. Taking max with the current l stops l from moving backwards when the old occurrence is already outside the window.

Approach

1. last = int[128] (ASCII), all 0. last[c] holds the index just after the most recent c.
2. l = 0, max = 0.
3. For r from 0 to n - 1: c = s.charAt(r).
4. l = max(l, last[c]) so the window skips past the previous c if it is inside the window.
5. max = max(max, r - l + 1).
6. last[c] = r + 1.
7. Return max.

Dry run on Example 1

- s = "abcabcbb", l = 0, max = 0
- r=0 'a': l=max(0,0)=0, max=1, last[a]=1
- r=1 'b': l=0, max=2, last[b]=2
- r=2 'c': l=0, max=3, last[c]=3
- r=3 'a': l=max(0,1)=1, len=3, max=3, last[a]=4
- r=4 'b': l=max(1,2)=2, len=3, last[b]=5
- r=5 'c': l=max(2,3)=3, len=3, last[c]=6
- r=6 'b': l=max(3,5)=5, len=2, last[b]=7

- r=7 'b': l=max(5,7)=7, len=1, last[b]=8
- Return max = 3

Java solution

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int[] last = new int[128];
        int l = 0, max = 0;
        for (int r = 0; r < s.length(); r++) {
            char c = s.charAt(r);
            l = Math.max(l, last[c]);
            max = Math.max(max, r - l + 1);
            last[c] = r + 1;
        }
        return max;
    }
}
```

Complexity

Time: O(n), each character processed once Space: O(1), a fixed array of 128

Edge cases handled

- Empty string: loop never runs, returns 0.
- Single space " ": answer 1 (spaces are characters too).
- "abba": at the last 'a', last[a]=1 but l is already 2, so max keeps l from going back.
- All characters distinct: answer is n.

Pattern to remember

Sliding window with a last-seen index: jump the left pointer with l = max(l, last[c]) instead of shrinking step by step.

4. Merge Intervals

Medium | #56 | Array, Sorting
https://leetcode.com/problems/merge-intervals/

Problem

Given a list of intervals [start, end], merge every group of overlapping intervals and return the non-overlapping intervals that cover exactly the same ranges. Intervals that only touch, like [1,4] and [4,5], also count as overlapping.

Examples

Example 1	
Input	`intervals = [[1,3],[2,6],[8,10],[15,18]]`
Output	`[[1,6],[8,10],[15,18]]`
Why	[1,3] and [2,6] overlap, so they become [1,6].

Example 2	
Input	`intervals = [[1,4],[4,5]]`
Output	`[[1,5]]`
Why	They touch at 4, so they are considered overlapping.

Example 3	
Input	`intervals = [[4,7],[1,4]]`
Output	`[[1,7]]`
Why	After sorting, [1,4] and [4,7] touch at 4 and merge.

Intuition

Once intervals are sorted by start, any interval that overlaps the current merged block must come right after it. So scan left to right: if the next start is at most the current end, stretch the current end; otherwise the block is finished and a new one starts.

Approach

1. Sort intervals by start.
2. cur = intervals[0], add cur to res.
3. For each interval in: if in[0] <= cur[1], set cur[1] = max(cur[1], in[1]) (cur is the same array stored in res, so res updates too).
4. Otherwise cur = in and add it to res.
5. Convert res to int[][] and return.

Dry run on Example 1

- intervals = [[1,3],[2,6],[8,10],[15,18]] (already sorted)
- cur = [1,3], res = [[1,3]]
- in = [1,3]: 1 <= 3, cur[1] = max(3,3) = 3
- in = [2,6]: 2 <= 3, cur[1] = max(3,6) = 6, res = [[1,6]]
- in = [8,10]: 8 > 6, cur = [8,10], res = [[1,6],[8,10]]
- in = [15,18]: 15 > 10, cur = [15,18], res = [[1,6],[8,10],[15,18]]
- Return [[1,6],[8,10],[15,18]]

Java solution

```
import java.util.*;
```

```
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int[]> res = new ArrayList<>();
        int[] cur = intervals[0];
        res.add(cur);
        for (int[] in : intervals) {
            if (in[0] <= cur[1]) {
                cur[1] = Math.max(cur[1], in[1]);
            } else {
                cur = in;
                res.add(cur);
            }
        }
        return res.toArray(new int[res.size()][]);
    }
}
```

Complexity

Time: O(n log n) for sorting, the scan is O(n) Space: O(n) for the result (plus sort space)

Edge cases handled

- Single interval: returned as is.
- Touching intervals [1,4],[4,5]: <= makes them merge.
- One interval fully inside another, e.g. [1,10],[2,3]: the max keeps end 10.
- Unsorted input like [[4,7],[1,4]]: sorting fixes it.
- Point intervals [5,5] and duplicates work the same way.

Pattern to remember

Interval problems: sort by start first, then a single pass comparing the next start with the current end.

5. Trapping Rain Water

Hard | #42 | Array, Two Pointers, Dynamic Programming, Stack, Monotonic Stack
https://leetcode.com/problems/trapping-rain-water/

Problem

height[i] is the height of a bar of width 1. After it rains, how many units of water are trapped between the bars?

Examples

Example 1	
Input	`height = [0,1,0,2,1,0,1,3,2,1,2,1]`
Output	`6`
Why	6 units of water are trapped in the dips between the bars.

Example 2	
Input	`height = [4,2,0,3,2,5]`
Output	`9`
Why	Water above each index: 0,2,4,1,2,0, total 9.

Intuition

Water above index i = min(highest bar on its left, highest bar on its right) - height[i]. With two pointers we do not need both maxima exactly. If height[l] < height[r], there is a bar on the right at least as tall as height[l], so the right side is not the limit for l; the water at l is decided only by lmax. The same logic works for r when the right side is smaller. So always move the smaller side.

Approach

1. l = 0, r = n - 1, lmax = rmax = ans = 0.
2. While l < r:
3. If height[l] < height[r]: lmax = max(lmax, height[l]); ans += lmax - height[l]; l++.
4. Else: rmax = max(rmax, height[r]); ans += rmax - height[r]; r--.
5. Return ans.

Dry run on Example 1

- height = [0,1,0,2,1,0,1,3,2,1,2,1], l=0, r=11
- h[0]=0 < h[11]=1: lmax=0, ans+=0, l=1
- h[1]=1 vs h[11]=1 (not <): rmax=1, ans+=0, r=10
- h[1]=1 < h[10]=2: lmax=1, ans+=0, l=2
- h[2]=0 < 2: lmax=1, ans+=1 -> ans=1, l=3
- h[3]=2 vs h[10]=2: rmax=2, ans+=0, r=9
- h[3]=2 vs h[9]=1: rmax=2, ans+=1 -> ans=2, r=8
- h[3]=2 vs h[8]=2: rmax=2, ans+=0, r=7
- h[3]=2 < h[7]=3: lmax=2, ans+=0, l=4
- h[4]=1: ans+=1 -> 3, l=5; h[5]=0: ans+=2 -> 5, l=6; h[6]=1: ans+=1 -> 6, l=7
- l == r = 7, stop. Return ans = 6

Java solution

```
class Solution {
    public int trap(int[] height) {
        int l = 0, r = height.length - 1;
        int lmax = 0, rmax = 0, ans = 0;
        while (l < r) {
            if (height[l] < height[r]) {
```

```
                lmax = Math.max(lmax, height[l]);
                ans += lmax - height[l];
                l++;
            } else {
                rmax = Math.max(rmax, height[r]);
                ans += rmax - height[r];
                r--;
            }
        }
        return ans;
    }
}
```

Complexity

Time: O(n), each index visited once Space: O(1)

Edge cases handled

- One or two bars: no space to hold water, answer 0.
- Strictly increasing or decreasing heights: answer 0.
- All bars equal: answer 0.
- Max size 2 * 10^4 with walls of 10^5 at both ends: total is about 2 * 10^9, which still fits in an int (int max is about 2.147 * 10^9).

Pattern to remember

Water at i = min(leftMax, rightMax) - h[i]. Two pointers: move the shorter side, because the shorter side's max is the real limit.